

**LAPORAN PRAKTIKUM
APLIKASI BERBASIS PLATFORM**

PERTEMUAN 9-MOBILE



Disusun oleh:

NAJWA HUMAIRAH

2311102134

S1 IF-11-04

Dosen Pengampu:

Cahyo Prihantoro, S.Kom., M.Eng

PROGRAM STUDI S1 TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2025/2026

LAPRAK PERTEMUAN 9

UI

DASAR TEORI

1. Container

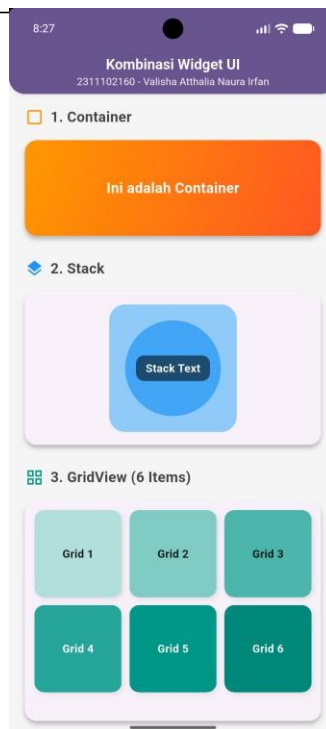
Container adalah salah satu widget layout yang paling umum dan paling sering digunakan di Flutter. Secara konseptual, Container adalah sebuah widget kenyamanan (convenience widget) yang menggabungkan fitur pengecatan (painting), penempatan letak (positioning), dan pengaturan ukuran (sizing). Jika sebuah Container tidak memiliki elemen anak (child), ia akan berusaha memenuhi ruang maksimum yang tersedia. Sebaliknya, jika memiliki child, ia akan mengecil (shrink-wrap) menyesuaikan ukuran anak tersebut, kecuali jika lebar atau tingginya diatur secara eksplisit. Properti penting pada widget ini meliputi padding untuk ruang kosong di dalam batas, margin untuk ruang kosong di luar batas, serta decoration yang memungkinkan pemberian gaya visual kompleks menggunakan BoxDecoration seperti warna solid, gradasi warna, radius sudut melengkung, hingga efek bayangan. Pada aplikasi ini, Container digunakan untuk membuat blok kotak utama dengan gradasi warna orange (LinearGradient) dengan penyesuaian tinggi tetap dan tepian yang melengkung.

```
Card( elevati  
on: 4,  
shape: RoundedRectangleBorder(borderRadius:  
BorderRadius.circular(15)),  
child:  
Container( heig  
ht: 120,  
decoration: BoxDecoration(  
borderRadius: BorderRadius.circular(15),  
gradient: const LinearGradient(  
colors: [Colors.orange, Colors.deepOrange],
```

```

        end: Alignment.bottomRight,
      ),
    ),
    child: const
      Center( child: Text(
        "Ini adalah Container",
        style: TextStyle(
          color: Colors.white,
          fontSize: 18,
          fontWeight: FontWeight.bold,
        ),
      ),
    ),
  ),
)

```



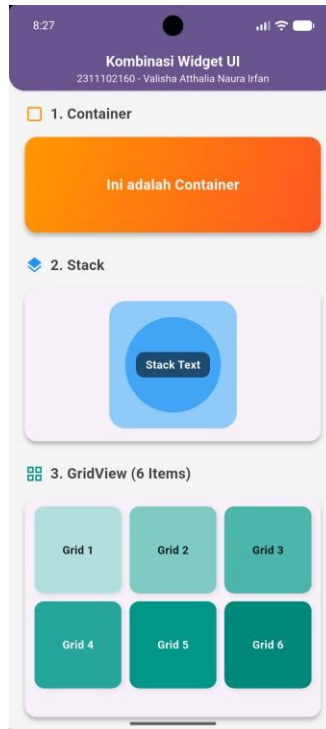
2. Stack

Stack adalah widget layout spasial yang memposisikan anak-anaknya secara absolut terhadap tepi kotaknya. Widget ini bekerja sangat mirip dengan pengaturan `position: absolute` pada CSS. Stack memungkinkan beberapa widget

digambar saling bertumpuk satu di atas yang lain pada sumbu Z (kedalaman layar). Mekanisme bertumpuknya bekerja di mana daftar anak (children) yang dimasukkan pertama kali pada array akan digambar paling bawah sebagai layer paling belakang, dan widget berikutnya akan digambar di atasnya. Secara default, Stack menempatkan anaknya di sudut kiri atas, namun melalui properti alignment, kita dapat memusatkan tumpukan tersebut. Pada aplikasi ini, Stack digunakan untuk membuat desain layering di mana sebuah wadah berbentuk kotak menjadi dasar, ditumpuk dengan wadah berbentuk lingkaran di tengahnya, dan ditumpuk lagi dengan kotak teks berisi tulisan di layer paling atas.

```
Card( elevati
  on: 4,
  shape: RoundedRectangleBorder(borderRadius:
BorderRadius.circular(15)),
  child: Padding(
    padding: const EdgeInsets.all(16.0),
    child: Center(
      child: Stack(
        alignment: Alignment.center,
        children: [
          Container(
            width: 160,
            height: 160,
            decoration:
              BoxDecoration( color:
Colors.blue.shade200,
borderRadius: BorderRadius.circular(20),
            ),
          ),
          Container(
            width: 120,
            height: 120,
```

```
        color: Colors.blue.shade400,
        shape: BoxShape.circle,
      ),
    ),
    Container(
      padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
      decoration: BoxDecoration(
        color: Colors.black54,
        borderRadius: BorderRadius.circular(10),
      ),
      child: const
        Text( "Stack
        Text", style:
        TextStyle(
          color: Colors.white,
          fontWeight: FontWeight.bold,
        ),
      ),
    ),
  ],
),
),
),
```



3. GridView

GridView adalah widget scrollable dua dimensi yang dirancang untuk menampilkan data dalam pola kisi-kisi atau grid yang memiliki baris dan kolom. GridView sangat berguna ketika kita ingin menampilkan banyak item visual secara efisien di ruang layar yang terbatas. Terdapat beberapa varian dari widget ini, salah satunya adalah `GridView.count` yang merupakan varian termudah untuk membuat grid di mana jumlah kolom sudah ditentukan secara statis. Varian lainnya adalah `GridView.builder` yang digunakan untuk membuat grid dinamis dalam jumlah besar secara on-demand. GridView memiliki properti `crossAxisSpacing` untuk memberikan jarak antar kolom, serta `mainAxisSpacing` untuk jarak antar baris. Dalam proyek ini, varian `GridView.count` digunakan dengan 3 kolom mendatar. Agar GridView dapat ditempatkan dengan aman di dalam sebuah `SingleChildScrollView` induk untuk mencegah konflik scroll vertikal, properti `physics` diatur menjadi `NeverScrollableScrollPhysics()` dan properti `shrinkWrap` diaktifkan agar tinggi grid menyesuaikan jumlah kontennya.

Card(elevati
on: 4,

```

    shape: RoundedRectangleBorder(borderRadius:
BorderRadius.circular(15)),
    child: Padding(
      padding: const EdgeInsets.all(12.0),
      child: GridView.count(
        shrinkWrap: true,
        physics: const NeverScrollableScrollPhysics(),
        crossAxisCount: 3,
        mainAxisSpacing: 10,
        crossAxisSpacing: 10,
        children: List.generate(6, (index)
        { return Container(
          decoration: BoxDecoration(
            color: Colors.teal[(index + 1) * 100],
            borderRadius: BorderRadius.circular(12),
            boxShadow: [
              BoxShadow(
                color: Colors.black.withOpacity(0.1),
                blurRadius: 4,
                offset: const Offset(2, 2),
              )
            ],
          ),
        ),
        child:
          Center( chil
            d: Text(
              "Grid ${index + 1}",
              style: TextStyle(
                fontWeight: FontWeight.bold,
                color: index > 2 ? Colors.white : Colors.black87,
              ),
            ),
          ),

```

```

);
}),
),
),
)

```



4. ListView (Statik)

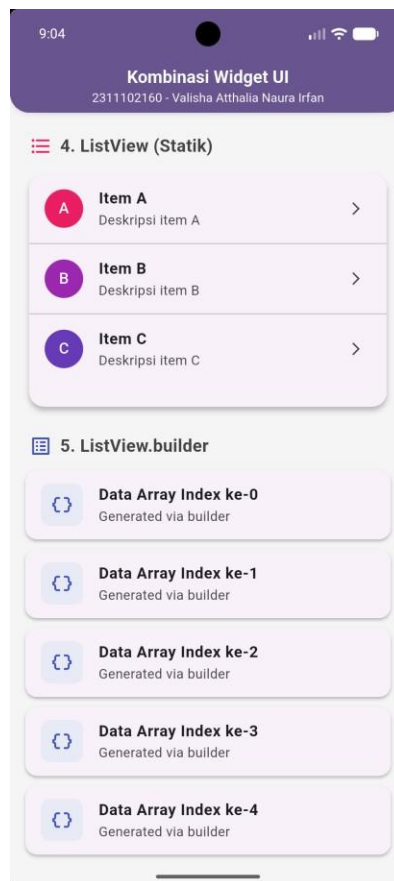
ListView adalah elemen fundamental di Flutter untuk menampilkan elemen yang tersusun dalam satu sumbu berurutan dan bisa digulir (scrollable). Mode statis ini digunakan saat jumlah data yang ditampilkan sedikit dan sudah diketahui sejak awal masa kompilasi. Pada ListView statis, seluruh widget anak yang terdaftar pada properti children akan dirender sekaligus di memori sehingga pendekatan ini tidak direkomendasikan untuk daftar item yang sangat panjang. Widget ini sering dikombinasikan dengan ListTile, yaitu widget Material Design standar yang memiliki slot khusus untuk elemen di kiri, elemen teks utama, teks tambahan, dan elemen di sebelah kanan. Pendekatan ini digunakan dalam aplikasi untuk merender list berukuran kecil berisi item secara langsung menggunakan struktur ListTile lengkap dengan komponen pembatas antar baris.

```
Card( elevation: 4,
  shape: RoundedRectangleBorder(borderRadius:
BorderRadius.circular(15)),
  child:
    ListView( shrinkWrap:
true,
physics: const NeverScrollableScrollPhysics(),
children: const [
  ListTile(
    leading:
      CircleAvatar( backgroundColor:
Colors.pink,
foregroundColor: Colors.white,
child: Text("A"),
    ),
    title: Text("Item A", style: TextStyle(fontWeight: FontWeight.bold)),
    subtitle: Text("Deskripsi item A"),
    trailing: Icon(Icons.arrow_forward_ios, size: 16),
  ),
  Divider(height: 1),
  ListTile(
    leading:
      CircleAvatar( backgroundColor:
Colors.purple, foregroundColor:
Colors.white, child: Text("B"),
    ),
    title: Text("Item B", style: TextStyle(fontWeight: FontWeight.bold)),
    subtitle: Text("Deskripsi item B"),
    trailing: Icon(Icons.arrow_forward_ios, size: 16),
  ),
  Divider(height: 1),
```

```

        backgroundColor: Colors.deepPurple,
        foregroundColor: Colors.white,
        child: Text("C"),
      ),
      title: Text("Item C", style: TextStyle(fontWeight: FontWeight.bold)),
      subtitle: Text("Deskripsi item C"),
      trailing: Icon(Icons.arrow_forward_ios, size: 16),
    ),
  ],
),
)

```



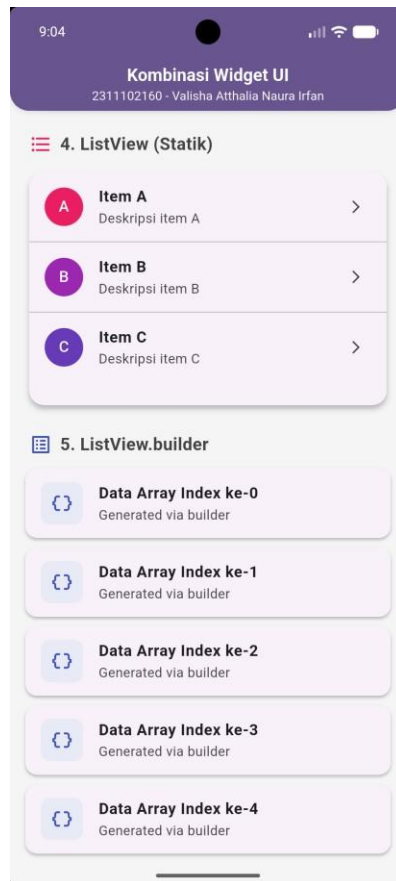
5. ListView.builder

Merupakan varian pembuatan ListView yang jauh lebih efisien dan modern, cocok digunakan untuk membangun daftar panjang berdasarkan data array atau objek dari API eksternal. Tidak seperti ListView statis yang membangun

seluruh daftarnya sekaligus, ListView.builder hanya membangun widget saat widget tersebut benar-benar akan terlihat di layar perangkat pengguna (on-demand). Saat pengguna melakukan scroll, item yang keluar dari layar akan dihancurkan, dan memori didaur ulang untuk membuat item baru yang masuk ke layar. Widget ini membutuhkan properti itemCount untuk menentukan batas perulangan dan itemBuilder sebagai fungsi pengembali widget yang sesuai dengan data pada index tertentu. Dalam proyek ini, builder diset untuk menghasilkan 5 item secara berulang di mana setiap baris bisa merespons perubahan angka indeks dari array.

```
ListView.builder( shrink
  Wrap: true,
  physics: const NeverScrollableScrollPhysics(),
  itemCount: 5,
  itemBuilder: (context, index)
    { return Card(
      elevation: 2,
      margin: const EdgeInsets.only(bottom: 10),
      shape: RoundedRectangleBorder(borderRadius:
BorderRadius.circular(12)),
      child: ListTile(
        leading: Container(
          padding: const EdgeInsets.all(10),
          decoration:
            BoxDecoration( color:
Colors.indigo.shade50,
              borderRadius: BorderRadius.circular(10),
            ),
        child: const Icon(Icons.data_object, color: Colors.indigo),
      ),
      title: Text(
        "Data Array Index ke-$index",
        style: const TextStyle(fontWeight: FontWeight.bold),
```

```
        subtitle: const Text("Generated via builder"),
      ),
    );
  },
)
```

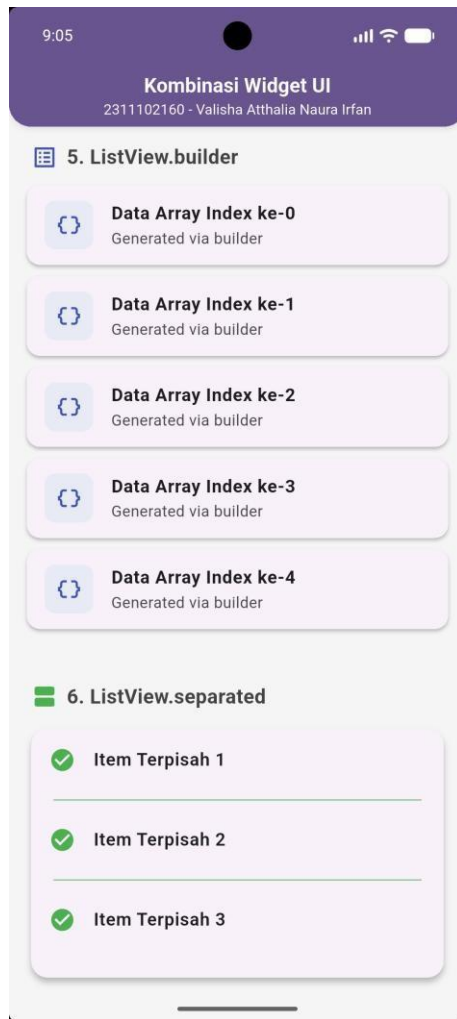


6. **ListView.separated**

Sebuah versi perluasan dari `ListView.builder` yang selain mampu membangun item secara dinamis, juga mampu menyisipkan elemen pemisah secara otomatis di antara setiap elemen data yang muncul. Widget ini sangat berguna jika kita memerlukan kontrol desain penuh terhadap bagaimana elemen-elemen di dalam list terpisah secara visual tanpa memunculkan pemisah tersebut sebelum item pertama atau setelah item terakhir. Penggunaannya memerlukan dua pembangun fungsional, yaitu `itemBuilder` untuk konten utamanya, dan `separatorBuilder` untuk elemen garis atau jarak pemisahannya. Aplikasi ini menggunakan properti `itemCount` sebanyak 3 di mana `itemBuilder`

mengembalikan desain baris teks utama, sedangkan separatorBuilder bertugas menggambar komponen pembatas berupa garis tebal berwarna di antara baris-baris tersebut.

```
Card( elevati
  on: 4,
  shape: RoundedRectangleBorder(borderRadius:
BorderRadius.circular(15)),
  child:
    ListView.separated( shrinkWr
      ap: true,
      physics: const NeverScrollableScrollPhysics(),
      itemCount: 3,
      separatorBuilder: (context, index) => const
        Divider( color: Colors.green,
          thickness: 1,
          indent: 20,
          endIndent: 20,
        ),
      itemBuilder: (context, index)
        { return ListTile(
          leading: const Icon(Icons.check_circle, color: Colors.green),
          title: Text(
            "Item Terpisah ${index + 1}",
            style: const TextStyle(fontWeight: FontWeight.w600),
          ),
        );
      },
    ),
```



LINK GITHUB : <https://github.com/najwahmrh/Webpro/tree/main/Pertemuan%209/ui>